Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación

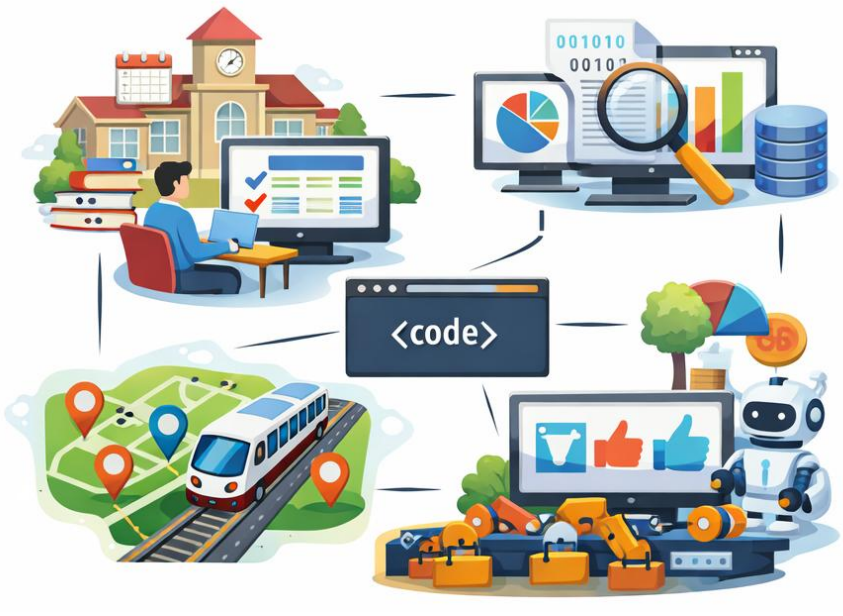
IIC2115 - Programación como Herramienta para la Ingeniería

Fundamentos: POO y EDD

Profesor: Hans Löbel

En este capítulo, ¿qué significará usar la programación como una herramienta?

- Los conceptos que veremos nos permitirán construir modelos computacionales de sistemas reales.
- Algunos ejemplos:
  - Gestión de cursos universitarios
  - Análisis de grandes volúmenes de datos
  - Modelamiento de redes de transporte
  - Sistemas de recomendación
  - Automatización de procesos
- Programar = representar sistemas y procesos en código



Agenda para hoy

- Cómo modelar sistemas con objetos
- Cómo organizar información con estructuras de datos
- Ejemplo guiado del capítulo
- Trabajo en notebooks

Cuando hablamos de un curso, ¿en qué estamos pensando?

- Un sistema puede representarse de distintas maneras.
- Cada representación es una abstracción del sistema real.
- Un modelo debe capturar:
  - Elementos relevantes del sistema
  - Propiedades de esos elementos
  - Interacciones entre ellos

Usemos el siguiente problema como ejemplo guía

Escriba un programa que permita ejecutar una serie de procesos docentes de fin e inicio de semestre para una institución de educación superior.

Ejemplos de procesos:

- Inscripción de estudiantes
- Asignación de secciones
- Verificación de cupos
- Cierre de semestre

¿Qué elementos existen en este sistema?

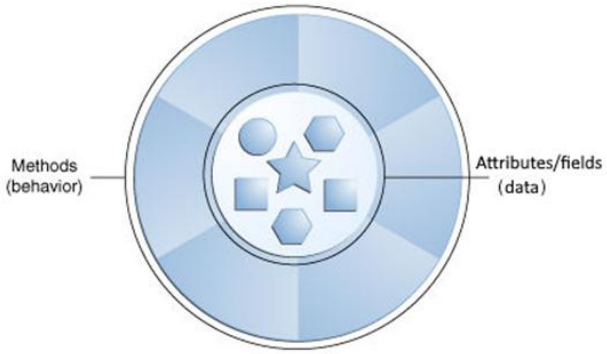
- Si queremos modelar este sistema, necesitamos identificar sus elementos principales.
- Ejemplos:
  - Estudiante
  - Curso
  - Sección
  - Profesor
  - Inscripción
- Cada uno de estos elementos puede modelarse como un objeto.

Objetos en software

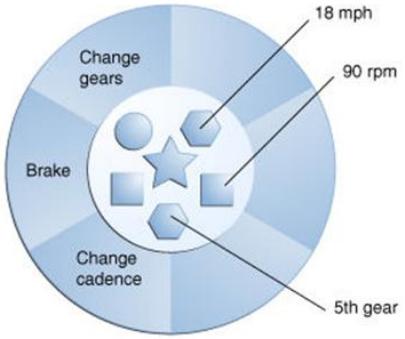
- Un objeto modela una entidad o abstracción dentro de un programa

- Un objeto está compuesto por:
  - Datos → describen su estado.
  - Comportamientos → acciones que pueden realizar

- Los datos se conocen como atributos.
- Los comportamientos se implementan como métodos.



(a) A software object



(b) Bicycle modelled as a software object

De objetos a clases

- Si muchos objetos comparten las mismas propiedades y comportamientos, necesitamos una plantilla común.
- Esa plantilla se llama clase.
- Ejemplo en el sistema:
  - Muchos estudiantes
  - Muchos cursos
  - Muchas secciones
- Conclusión: una clase define la estructura de un tipo de objeto.

Clase vs Objeto

Una clase:

- Define atributos
- Define métodos

Un objeto:

- Es una instancia de una clase
- Tiene valores específicos en sus atributos

```python
class Estudiante:

    def __init__(self, id, nombre):
        self.id = id
        self.nombre = nombre
        self.cursos = []

estudiante1 = Estudiante(1, "Ana")
estudiante2 = Estudiante(2, "Luis")

type(estudiante1)
```

```
<class '__main__.Estudiante'>
```

Extendamos la clase Estudiante agregando comportamiento

- Los métodos representan acciones del objeto
- En este caso, un estudiante puede inscribir un curso

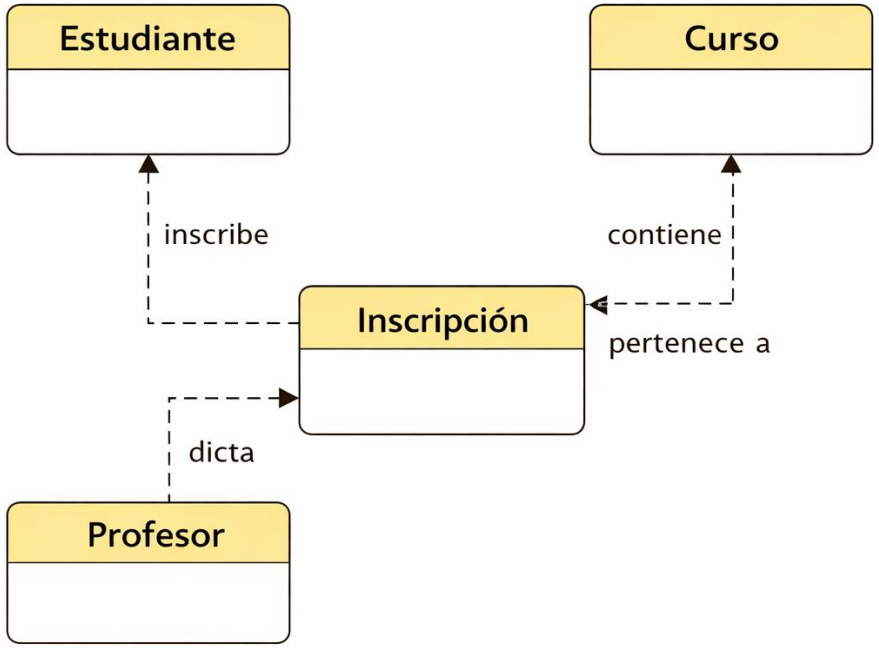
```python
class Estudiante:

    def __init__(self, id, nombre):
        self.id = id
        self.nombre = nombre
        self.cursos = []

    def inscribir(self, curso):
        self.cursos.append(curso)
```

Los objetos no existen aislados

- En un sistema real, los objetos interactúan entre sí.
- En nuestro ejemplo:
  - Estudiantes
  - Cursos
  - Profesores
  - Inscripciones
- El comportamiento del sistema surge de las interacciones entre objetos.
- Programar sistemas implica siempre modelar las interacciones entre objetos



Continuemos modelando el sistema

```python
class Curso:

    def __init__(self, codigo, nombre, cupo):
        self.codigo = codigo
        self.nombre = nombre
        self.cupo = cupo
        self.estudiantes = []

    def agregar_estudiante(self, estudiante):
        self.estudiantes.append(estudiante)
```

```python
est1 = Estudiante(1, "Ana")
curso = Curso("IIC2115", "Programación", 100)

est1.inscribir(curso)
curso.agregar_estudiante(est1)
```

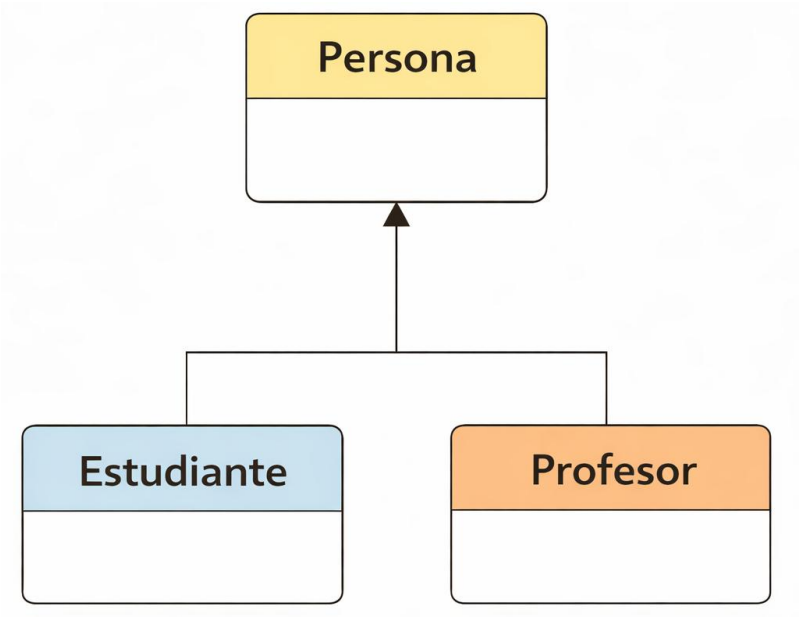
¿Qué problema tiene la modelación de la interacción entre objetos de estas dos clases?

Relaciones entre clases

- Las clases pueden relacionarse de distintas formas.
- Acá cubriremos 4:
  - Herencia
  - Polimorfismo
  - Agregación
  - Composición
- Estas relaciones permiten construir modelos más complejos y sofisticados.

Herencia se relaciona con la especialización

- La herencia permite definir una clase a partir de otra.
- Se usa cuando una clase es una especialización de otra.
- Esto permite:
  - Reutilizar código
  - Representar jerarquías naturales en el sistema



Herencia se relaciona con la especialización

```python
class Persona:

    def __init__(self, id, nombre):
        self.id = id
        self.nombre = nombre
```

```python
class Estudiante(Persona):

    def __init__(self, id, nombre):
        super().__init__(id, nombre)
        self.cursos = []
```

```python
class Profesor(Persona):

    def __init__(self, id, nombre, departamento):
        super().__init__(id, nombre)
        self.departamento = departamento
```

Polimorfismo

- Distintas clases en una misma jerarquía pueden responder a una misma operación.
- Pero cada una puede implementar esa operación de manera distinta.
- Por ejemplo, cada miembro de la comunidad académica podría mostrar en pantalla de manera distinta su información.

```
class Persona:

    def mostrar_info(self):
        return "Persona del sistema"
```

```
class Estudiante(Persona):

    def mostrar_info(self):
        return "Estudiante inscrito en cursos"
```

```
class Profesor(Persona):

    def mostrar_info(self):
        return "Profesor del sistema"
```

Agregación

- Una clase puede contener objetos de otra clase
- Por ejemplo, un curso contiene estudiantes inscritos.
- Pero, un estudiante puede existir independiente del curso.
- Esta relación se conoce como agregación.

Composición

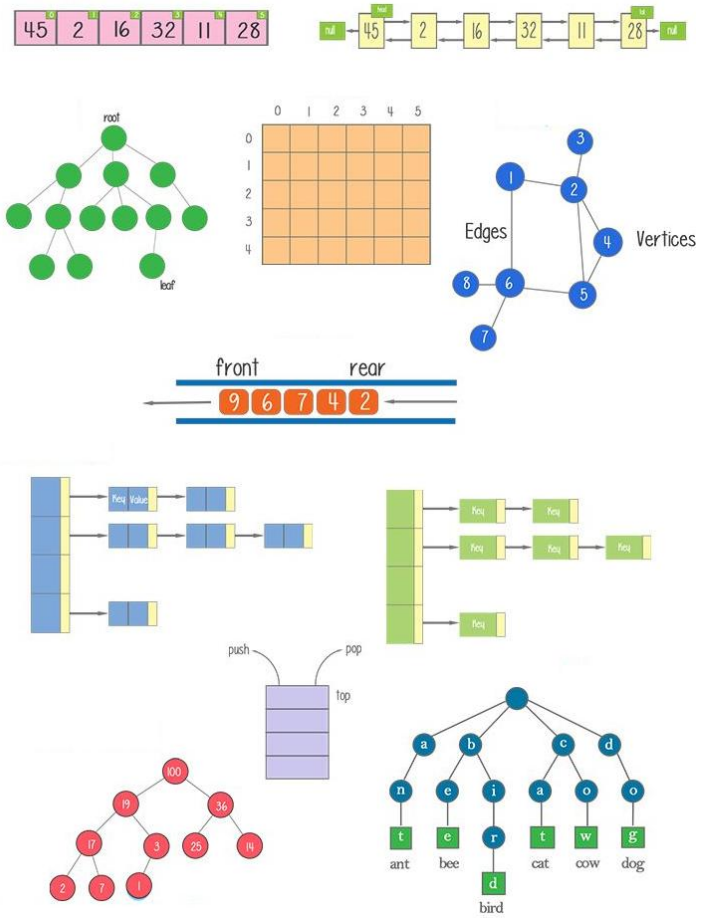
- En algunos casos, un objeto no tiene sentido fuera de otro.
- Por ejemplo, un objeto de la clase Inscripción
- Una inscripción existe solo si también existen:
  - Un estudiante
  - Un curso
- Esta relación se conoce como composición.

¿Dónde aparecen las estructuras de datos?

- Para implementar este modelo, necesitamos almacenar y acceder de manera eficiente a:
  - Estudiantes
  - Profesores
  - Cursos
  - Inscripciones
- Para esto utilizaremos estructuras de datos, principalmente:
  - Listas y tuplas
  - Diccionarios
  - Árboles

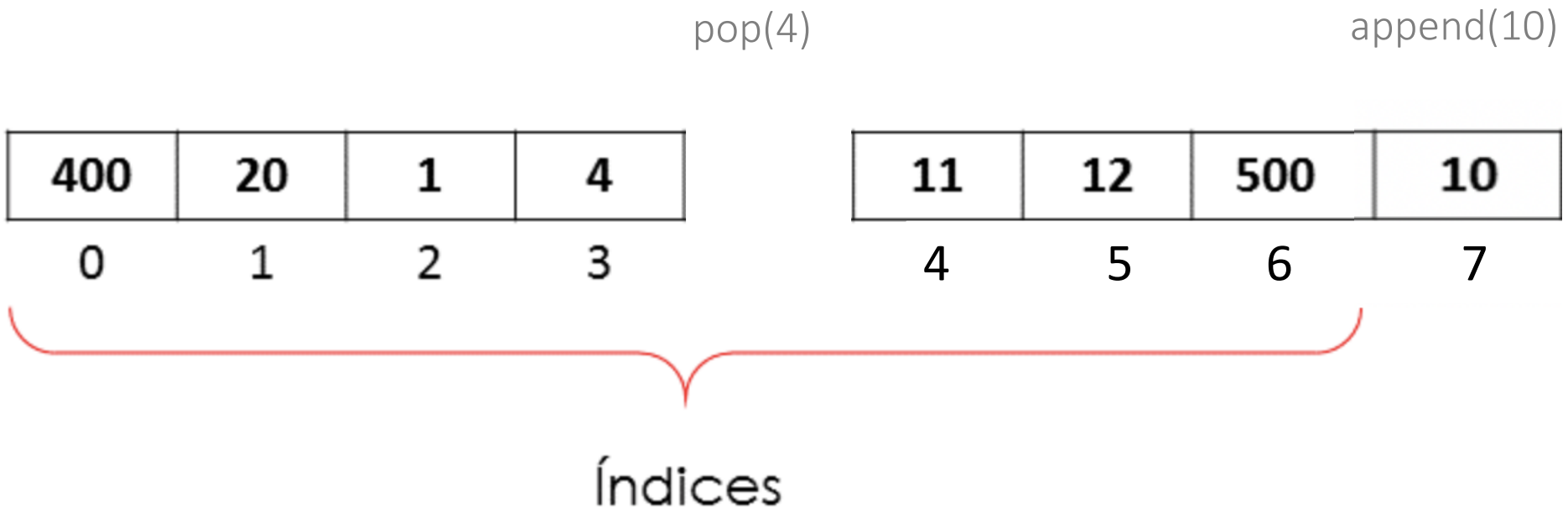
Qué serán para nosotros las estructuras de datos

- Las estructuras de datos (incluyendo clases) son tipos de datos especializados para organizar y acceder a información de forma eficiente.
- Elegir la estructura adecuada o organizar bien las clases es clave para desarrollar buenos programas y resolver problemas de forma realista.
- Conviene partir con una solución simple y luego incorporar estructuras de datos cuando sea necesario.



Listas

- Las listas son estructuras que guardan datos de forma ordenada.
- Son mutables (modificables).



Tuplas

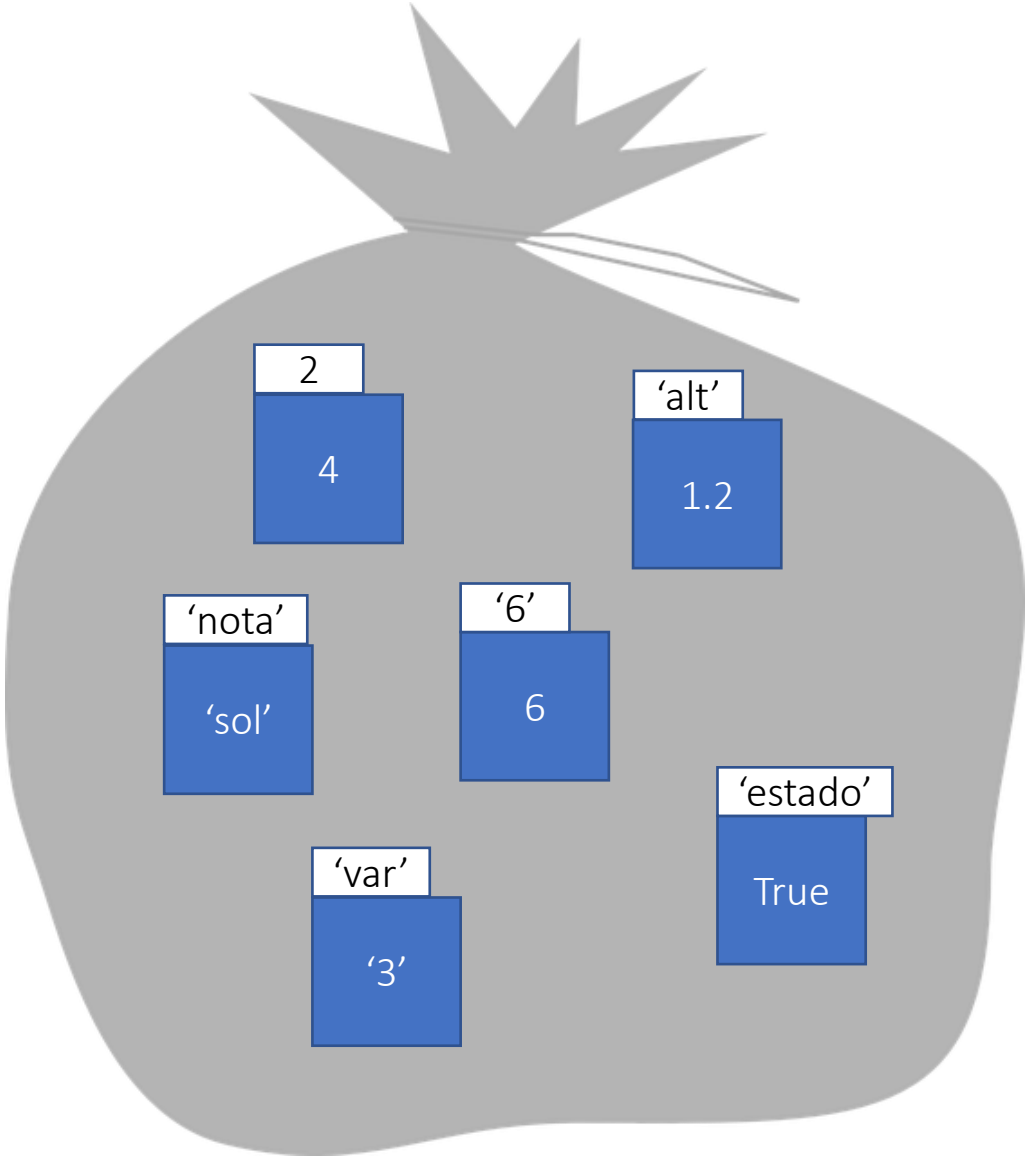
- Similares a las listas, permiten manejar datos de forma ordenada.
- Al igual que las listas, se accede a los datos mediante índices basados en el orden que fueron ingresados.
- A diferencia de las listas, son inmutables.

400	20	1	4	10	11	12	500
0	1	2	3	4	5	6	7

Índices

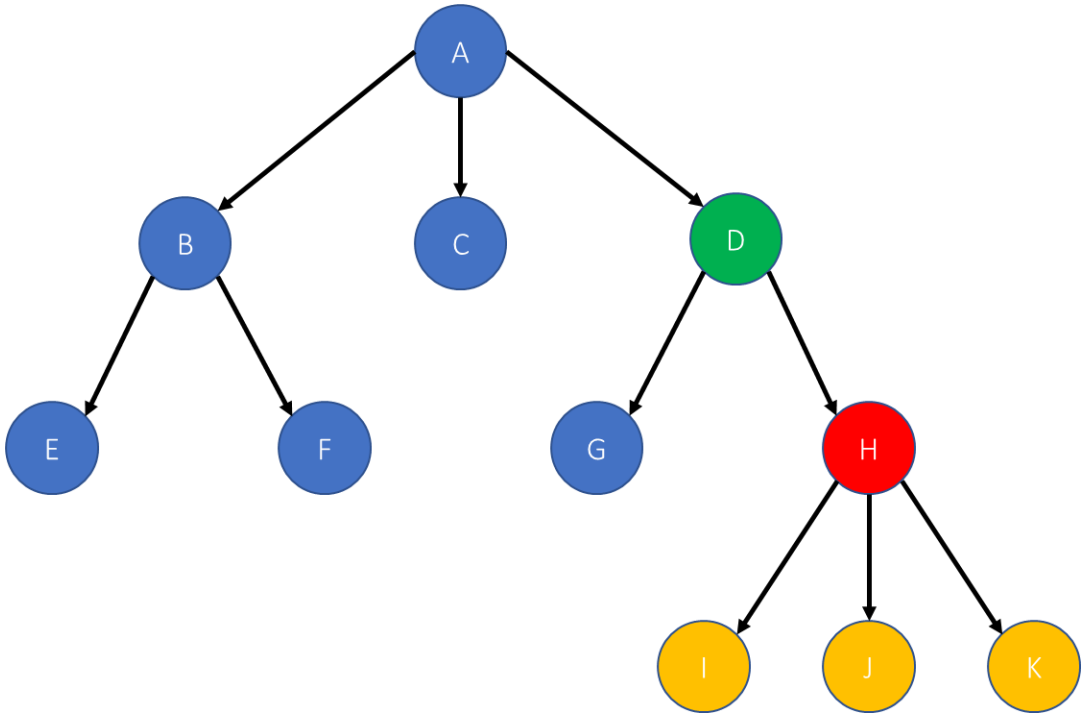
Diccionarios

- Permiten almacenar datos basados en una asociación de pares de elementos, a través de una relación llave-valor.
- Acceso a valores a través de la llave es instantáneo, no se necesita realizar una búsqueda (análogo a un índice).
- Se prefiere a una lista cuando el caso de uso más común no implica revisar todos los elementos, sino solo algunos fácilmente encontrables a través de la llave.



Árboles

- Son útiles cuando los datos tienen una estructura jerárquica.
- En nuestro ejemplo, los árboles nos permitían construir:
  - Una herramienta de seguimiento curricular: prerrequisitos vienen antes que requisitos en el árbol.
  - Un sistema de búsqueda de cursos con autocompletar.



Para cerrar, hablemos un poco sobre el modo de trabajo

- División progresiva del problema en subproblemas más simples: dividir y conquistar
- Dada la temática del capítulo, el foco de la división debe ponerse en las clases y la interacción de los objetos
- Dado que usaremos Python Notebooks, la subdivisión debe considerar adecuadamente la estructura de celdas.

Para cerrar, hablemos un poco sobre el modo de trabajo

Recomendaciones/Exigencias:

- Mantener una celda temáticamente coherente (familia de clases, clase y test, etc).
- No usar una única celda con todas las clases.
- Partir con clases que modelan entidades evidentes, luego pasar a clases que manejen la coordinación.
- Usar diagramas siempre que sea posible
- Siempre poner por escrito los supuestos y simplificaciones.

Cómo sigue la sesión de hoy

- Breve revisión de los ejercicios
- Trabajo en los ejercicios
- Entrega final del avance en repositorios y ticket de salida (17:10 a 17:30)

Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación

IIC2115 - Programación como Herramienta para la Ingeniería

Fundamentos: POO y EDD

Profesor: Hans Löbel